

Project Report

5G teleoperated Robotic Arm

David Rajchman

Table of contents

1	Introduction	2
2	Project assignment	2
2.1	Approach to solving the assignment	2
2.2	Additional devices	2
2.3	ROS2 modular offloading codebase	3
2.4	Original network topology	4
2.5	Modular Gateway	5
2.6	P2P mode - New	5
2.7	P2P with discovery	6
2.8	ROARM Manufacturer ROS2 Demo	6
2.9	Custom GUI	7
2.10	5G network	8
2.11	Performance	8
2.12	Future steps	9
3	Demo time	9
3.1	Hardware Demonstration	9
3.2	P2P mode setup	10
3.3	P2P DS mode setup	11

1 Introduction

The project is meant

2 Project assignment

The main goal of the project was for the author to familiarize themselves with the ROARM-M1 robotic arm, and create a solution that would allow for remote control of the robotic arm over the EURECOM 5G network. The author was expected to use the ROS2 toolchain for the robotic arm control.

2.1 Approach to solving the assignment

The author was given access to a raspberry pi 4 and a lab laptop running ubuntu 22 lts for the use in his project. The following connection scheme was created to allow for the assignment completion Figure 1 .

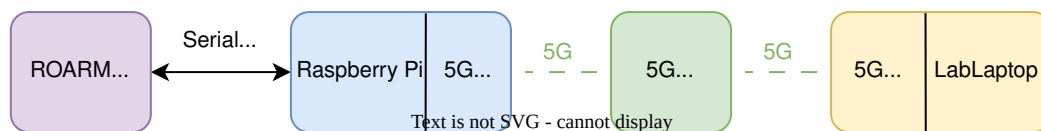
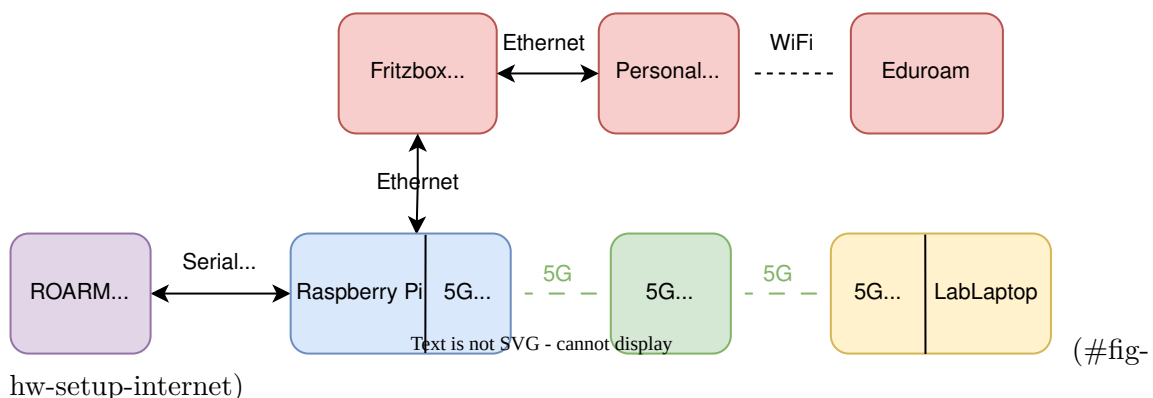


Figure 1: The physical elements of the solution

To make the completion of the project assignment more straightforward, it was dissected into 3 distinct parts. 1) Connect the robotic arm to the ROS2 environment and allow for its control with the gamepad 1) Allow for forwarding of the control topics between two ros2 instances 1) Connect the arm-side ROS2 instance with the controller-side ROS2 instance together using the EURECOM 5G network

2.2 Additional devices



hw-setup-internet)

To allow for the configuration of the raspberry pi it had to be given internet access. Initially use of the lab LAN network was considered, however the ports have a MAC filter on them, which made connection of the PI complicated. As a second option the author used his personal laptop which had access to the EURECOM eduroam network. Then the author forwarded the network to the RPI. By default the ubuntu 24 LTS for raspberry does not allow for a direct connection between a laptop and the RPI, this is something that can be configured, but without ssh access, the configuration would have to be done by adjusting system files on the boot sdcard. This is a process which is prone to errors and very hard to debug. For this reason a router was used as a DHCP server and a switch and both the RPI and the personal laptop were connected to it. Then the RPI simply used the laptop IP as the default gateway. (configuration script available in appendix XXX). This configuration shown in Image ?@fig-hw-setup-internet also allows for reliable SSH access to the RPI. One last issues was the time, the RPI did not sync its time through NTP when its RTC time was greatly different from real time, this could be mitigated by manually setting approximately correct time and then the NTP would sync the precise time. A simple script was created to automatically get approximately accurate time from the internet

2.3 ROS2 modular offloading codebase

The author of this project has worked on another project with similar use in the 6G lab at FEE CTU. The resulting codebase fully written by the author of this project was used as a starting point for the ros2 topic forwarding in this project. This chapter will briefly describe the original codebase and its purpose to make the final project codebase easier to understand.

The goal of the 6G lab assignment was to create a solution that could forward data from ros2 topic on one ros2 instance to another ros2 instance. The intended usecase was computation offloading from vehicles to mec servers. The solution had to be highly flexible, as one of the requirements was that a single vehicle had to be able to offload to at least 2 different MEC server at one time with every MEC server working on a different task. The MEC server were clones of the vehicle ROS2 environment. There are existing solutions allowing for connection of 2 existing ros2 environments together utilising the ros2 DDS protocol. However none of them provide this level of flexibility.

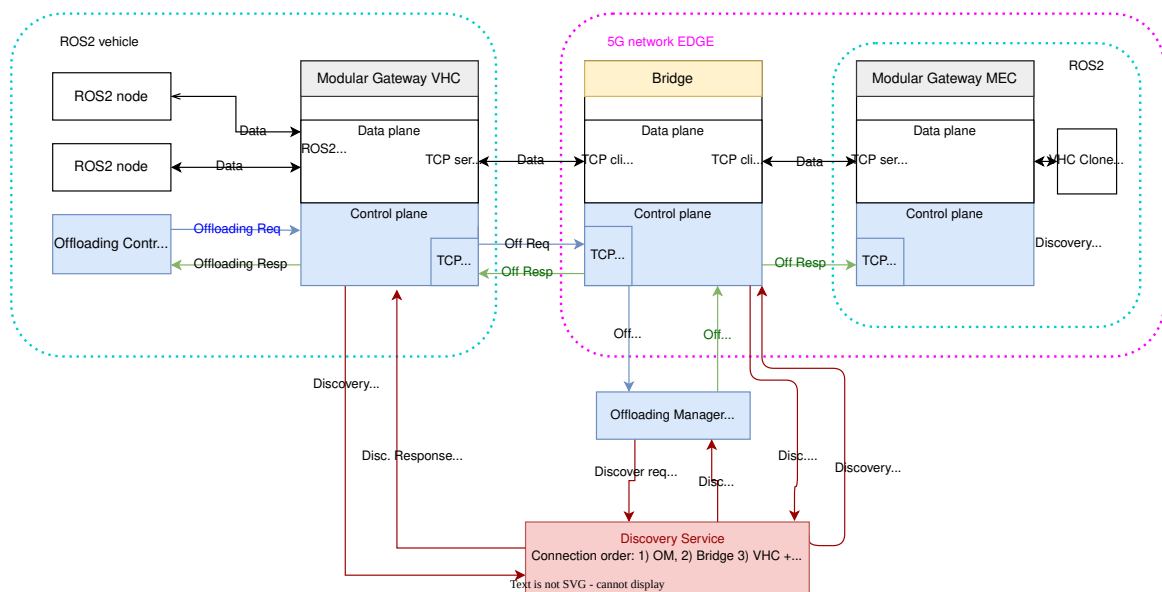
For this reason an architecture shown in @fig_om_network was designed. The main components were the modular gateway, bridge and offloading manager. The offloading manager held a configurable algorithm that distributed limited resources to the vehicles, by approving or rejecting the offloading requests. If the request was approved, the bridge would create an internal route for the offloaded topics between the vehicle and the assigned MEC server. The data flowed in a separate connection and in all components there was a clear distinction between data plane and control plane parts.

The modular gateway shown in @fig_mqw is the core part of the solution that transmits the data from ros2 topics towards the 5G network. Each topic has its own handler, this handler does ROS2 serialization and deserialization, and also allows for data manipulation for

example removing unnecessary parts of the message. To achieve highest possible performance the modular gateway is multithreaded.

- My previous assignment, similar use-case
- Created by me during my stay at the 6G Lab at FEE CTU
- The assignment:
 - Transparent ROS2 to ROS2 topic transmission
 - Per topic/task MEC server allocation
 - Adjustable offloading resource allocation algorithm
 - Do not use high level tools/frameworks (e.g. MQTT)
 - Low latency, high throughput

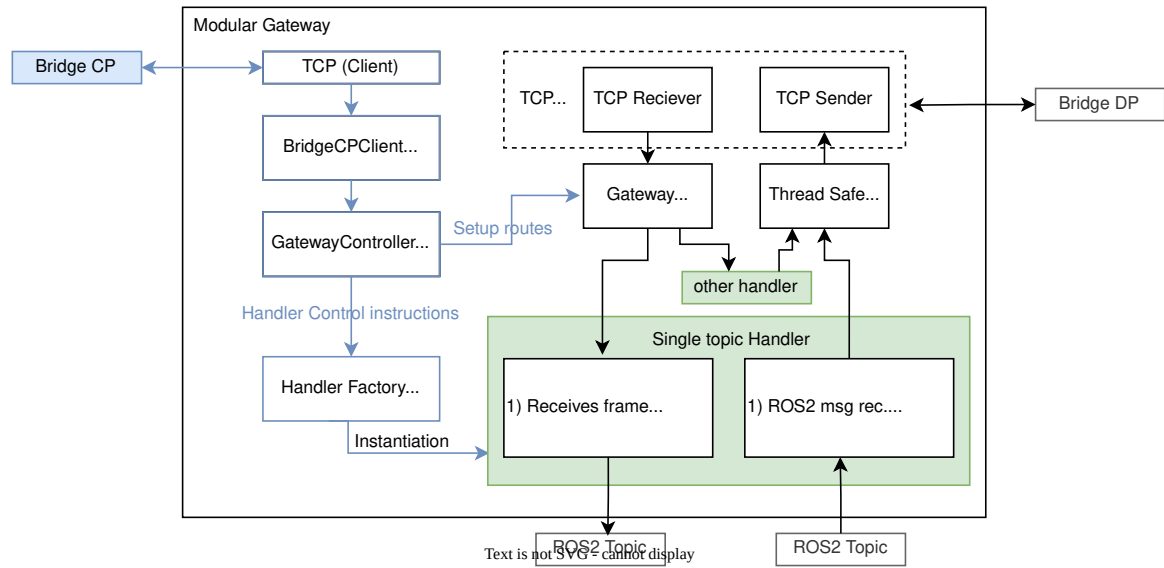
2.4 Original network topology



{fig_om_net-

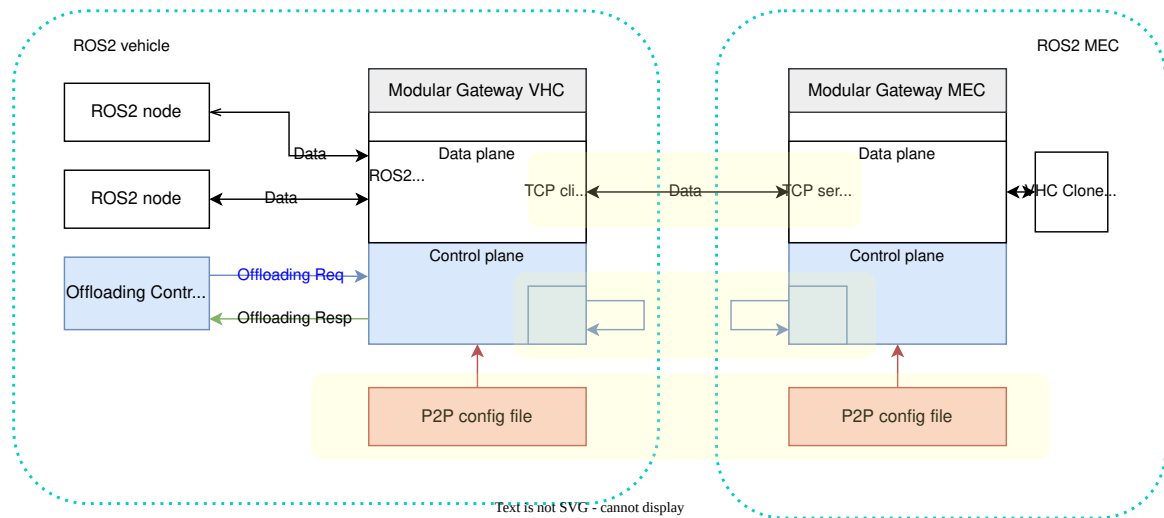
work}

2.5 Modular Gateway

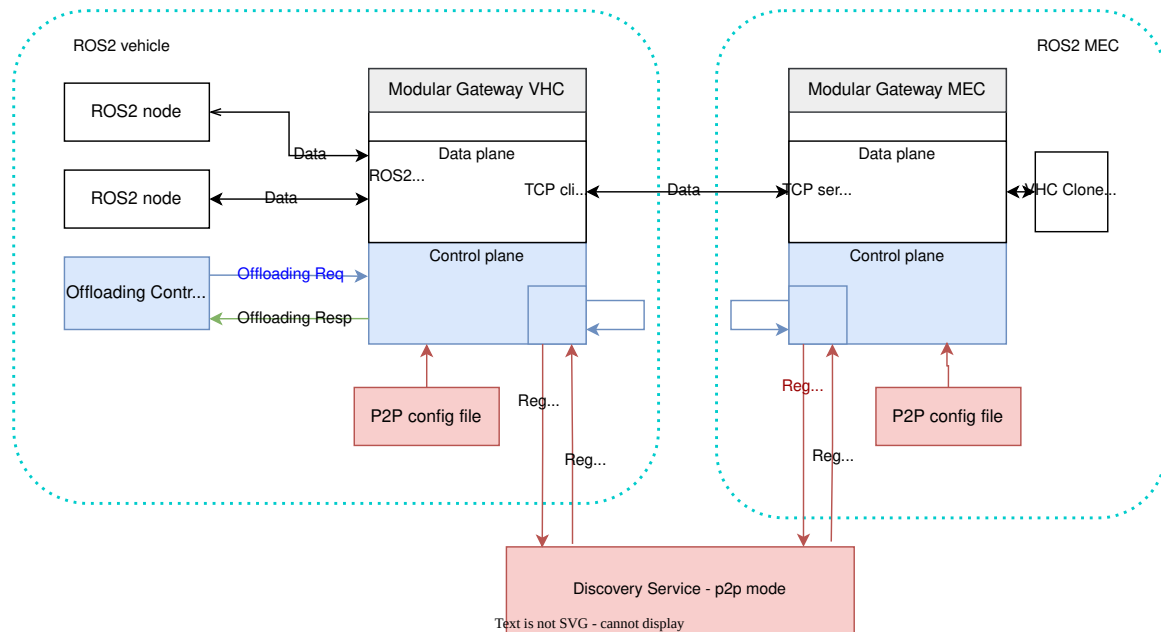


{fig_mqw}

2.6 P2P mode - New

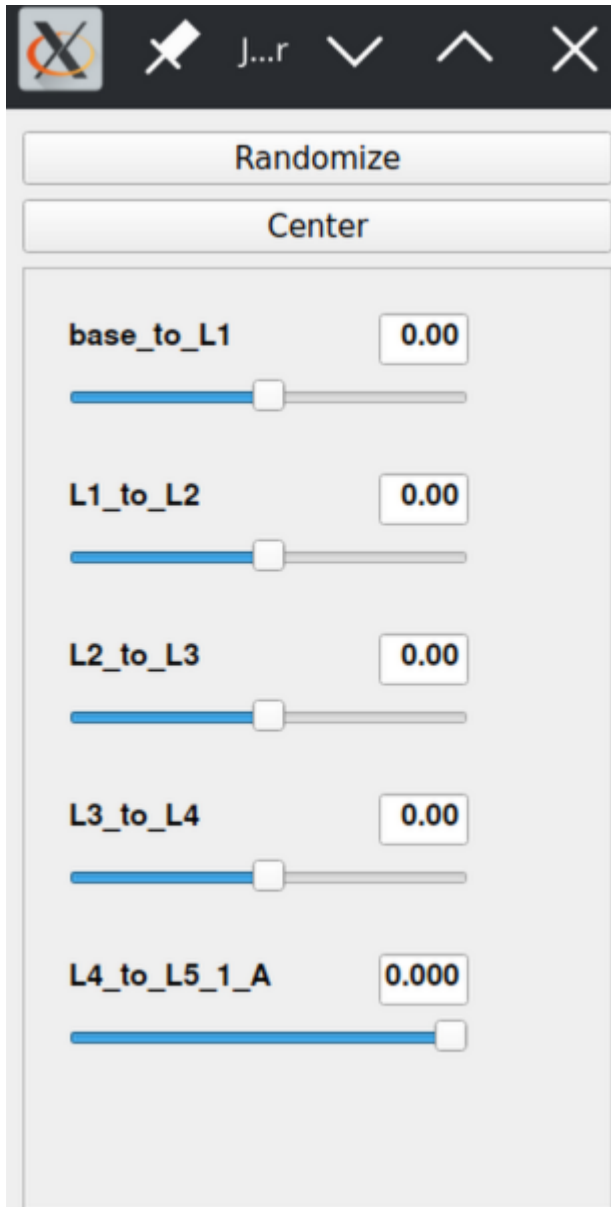


2.7 P2P with discovery



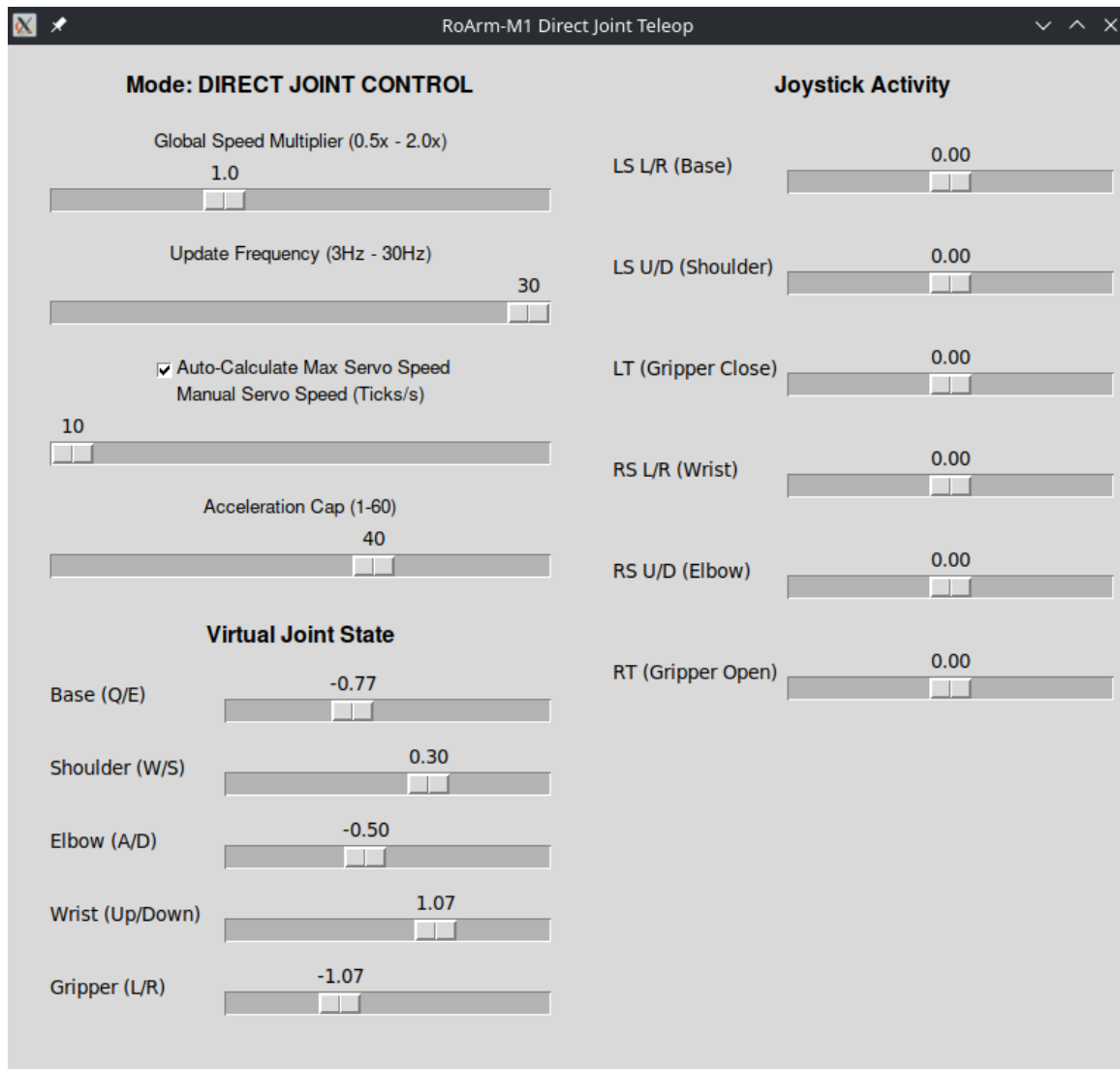
2.8 ROARM Manufacturer ROS2 Demo

- **joint_state_publisher_gui Node:**
 - Basic slider to manipulate each joint
- **/joint_states Topic:**
 - Intended joint coordinates (radian angle)
- **serial_ctrl Node:**
 - Subscribes to `/joint_states`,
 - Radians -> hardware servo values
 - Connects to the arm over serial



2.9 Custom GUI

- Replaces `joint_state_publisher_gui`
- ROS2 joy node for gamepad support
- Keyboard control
- Adjustable
 - Speed
 - Joint Acceleration
 - Update frequency



2.10 5G network

- 5G OAI network created by Meriem
- Severe issues connecting
 - Probably due to interference
 - Only during working hours (stable after 5:30 pm)
- RTT 40ms between laptop and RPI
- RTT 50ms to Google DNS

2.11 Performance

- ROS2 forwarding on localhost - ROS2 to ROS2 RTT ~ 1 ms

- 5G ROS2 to ROS2 RTT ~ 40 ms with jitter
- Precise arm control achievable
- Best settings in the GUI for smoothest motion:
 - 15 Hz update frequency
 - 20 steps/s² maximum acceleration (60 is default)

2.12 Future steps

- Slicing support
- DiscoveryService NAT traversal using STUN
- Further firmware and driver optimisation
 - Maximum movement execution time
 - Constant latency mode

3 Demo time

3.1 Hardware Demonstration

<https://youtu.be/R55aSlyh2z4>

3.2 P2P mode setup

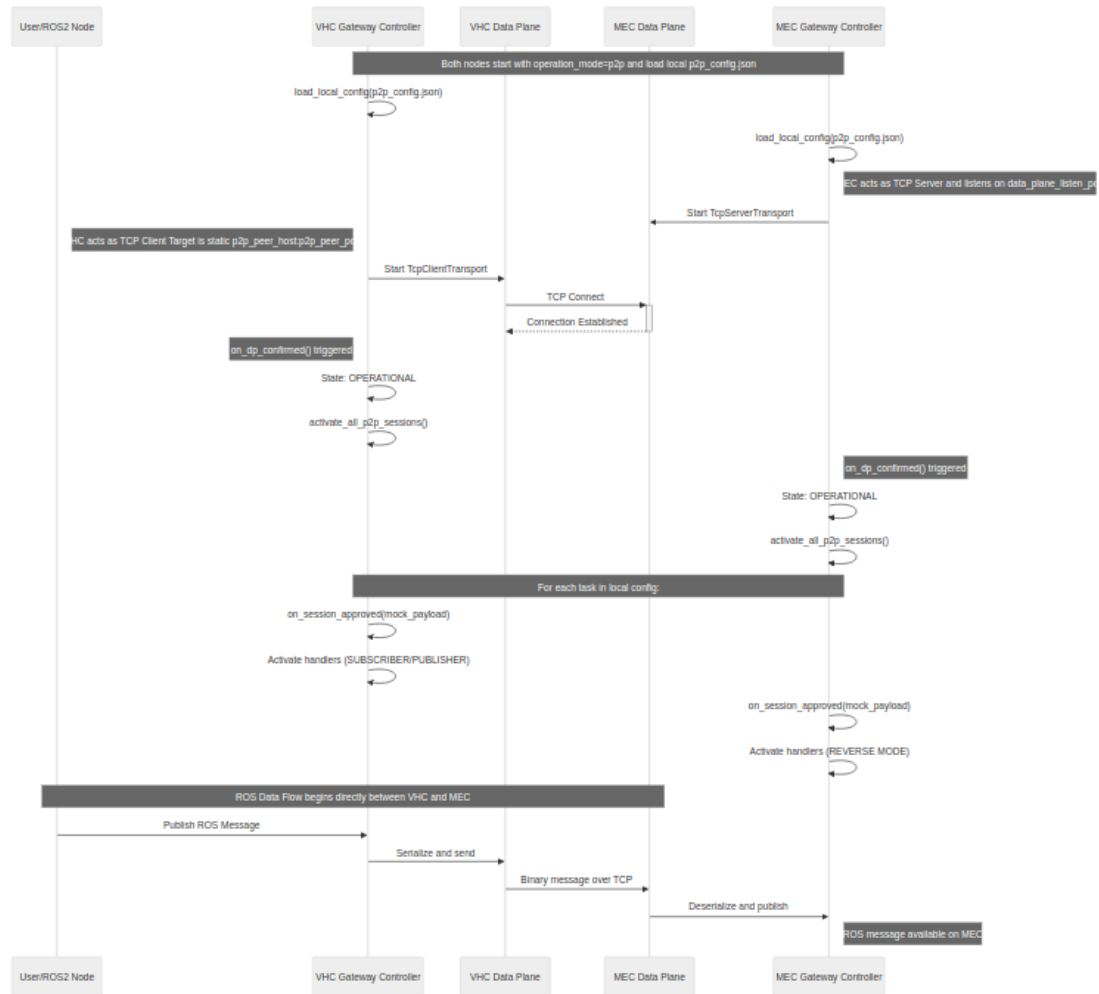


Figure 2: P2P mode setup sequence diagram

3.3 P2P DS mode setup

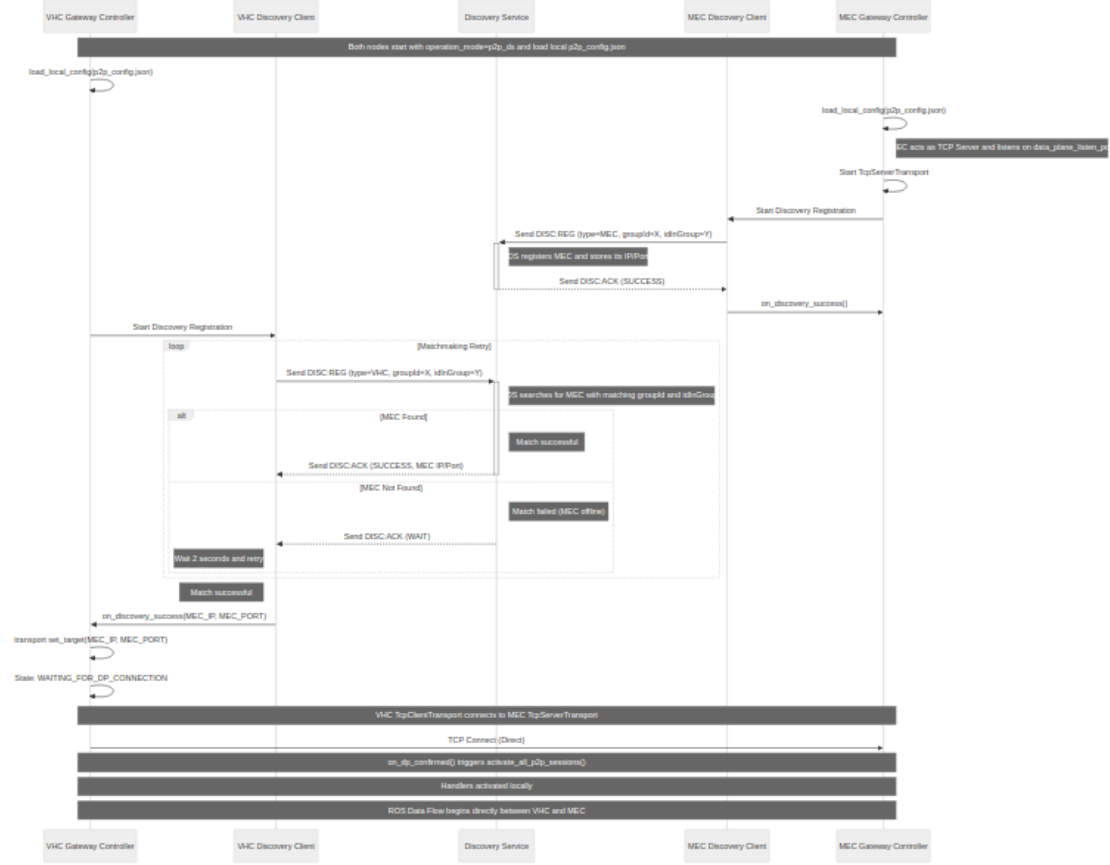


Figure 3: P2P DS mode setup sequence diagram